Ingeniería en Tecnologías de la Información y las Comunicaciones

Tecnologías Inalámbricas

12:00 p m

Evaluativo 2 - NRF24l01 - Sistema de control

Profesor: Ricardo Alejandro Rodríguez Jiménez

Equipo:

Ramírez Rodríguez Martín de Jesús 23151193

Santacruz Rodarte Cristian Alexis 22150108

Pedroza Pérez Emmanuel 23151202

Fecha: 23 de marzo de 2026

Índice

Introducción 3

Descripción de la Esp32 4

Descripción de la NRF24L01 4

Diferenciación entre las NRF24L01 con y sin PA+LNA con y sin antena 5

Descripción del funcionamiento del sensor elegido 5

Descripción del actuador elegido 7

Planteamiento del proyecto 7

Circuitos 9

1. Transmisor 9

a) Diseño 9

b) Código 10

2. Receptor 14

c) Diseño 14

d) Código 15

Conclusiones 21

Enlace a Video Demostración 21

Enlace a GitHub 21

Introducción

En continuación con la comunicación entre dos ESP32 empleando un sensor ultrasónico, se busca establecer un enlace confiable de transmisión y recepción de datos, en el que el dispositivo receptor confirme la conexión y muestre periódicamente las mediciones obtenidas. Además de evaluar el alcance máximo de la señal, el proyecto incorpora protocolos de encriptación y autenticación que garantizan la confidencialidad, integridad y legitimidad de la información transmitida, evitando riesgos como la interceptación o la suplantación de identidad. De esta manera, se integra la adquisición de datos físicos con la seguridad informática, fortaleciendo la confiabilidad del sistema embebido.

Descripción de la Esp32

La ESP32 es un microcontrolador versátil con Wi-Fi y Bluetooth integrados:

Características principales:

Procesador dual-core Tensilica LX6 a 240 MHz.

Conectividad Wi-Fi (802.11 b/g/n) y Bluetooth/BLE integrada.

Amplia cantidad de GPIOs para sensores, actuadores y periféricos.

Soporte para protocolos como SPI, I2C, UART, PWM y ADC.

Ventajas: Ideal para proyectos IoT, bajo costo, gran comunidad de soporte y capacidad de ejecutar tareas complejas con conectividad inalámbrica.

Descripción de la NRF24L01

NRF24L01 es un transceptor de 2.4 GHz diseñado para comunicaciones de bajo costo y bajo consumo.

Tipo: Transceptor inalámbrico de 2.4 GHz.

Funciones:

Comunicación bidireccional (envío y recepción).

Velocidades configurables: 250 kbps, 1 Mbps y 2 Mbps.

Bajo consumo de energía, adecuado para dispositivos portátiles.

Uso típico: Comunicación entre microcontroladores (Arduino, ESP32, STM32) en proyectos de domótica, sensores inalámbricos y control remoto.

Diferenciación entre las NRF24L01 con y sin PA+LNA con y sin antena

La versión con PA+LNA y antena externa ofrece mayor alcance y estabilidad que la versión estándar.

Versión	PA (Power Amplifier)	LNA (Low Noise Amplifier)	Antena	Alcance aproximado	Consumo
NRF24L01 estándar (sin PA/LNA, sin antena)	No	No	PCB integrada	10–30 m en interiores	Muy bajo
NRF24L01 con antena externa (sin PA/LNA)	No	No	Antena SMA	50–100 m	Bajo
NRF24L01 con PA+LNA y antena externa	Sí	Sí	Antena SMA	500–1000 m en condiciones óptimas	Más alto

PA (Power Amplifier): Incrementa la potencia de transmisión, extendiendo el alcance.

LNA (Low Noise Amplifier): Mejora la sensibilidad de recepción, reduciendo interferencias.

Antena externa: Proporciona mejor ganancia y estabilidad frente a obstáculos.

Descripción del funcionamiento del sensor elegido

Sensor Ultrasónico HC-SR04

El HC-SR04 funciona midiendo el tiempo que tarda un pulso ultrasónico en viajar hasta un objeto y regresar como eco.

1. Disparo del pulso (Trigger):
   - El ESP32 envía una señal de 10 µs al pin Trigger del sensor.
   - Esto hace que el HC-SR04 emita un tren de ondas ultrasónicas a 40 kHz.

2. Reflexión del sonido:
   - Las ondas viajan por el aire hasta chocar con un objeto.
   - El eco rebota y regresa al sensor.

3. Recepción del eco (Echo):
   - El pin Echo del sensor se pone en alto mientras espera la señal reflejada.
   - Cuando el eco regresa, el pin Echo se pone en bajo.
   - El tiempo que estuvo en alto corresponde al viaje de ida y vuelta del sonido.

Cálculo de distancia:

- El ESP32 mide ese tiempo en microsegundos.

$$Distancia = \frac{Tiempo(0.0343)}{2}$$

La distancia se mide en centímetros y el tiempo en segundos.

Donde 0.0343 cm/µs es la velocidad del sonido en aire.

El resultado se divide entre 2 porque el sonido hace un recorrido de ida y vuelta.

En la práctica, el HC-SR04 te da lecturas entre 2 cm y 400 cm con una precisión de unos ±3 mm, lo que lo hace muy útil para proyectos de detección de obstáculos o medición de distancias.

Descripción del actuador elegido

LED (Diodo Emisor de Luz)

El LED (diodo emisor de luz) es un componente electrónico que transforma energía eléctrica en energía lumínica.

Es un semiconductor que emite luz cuando circula corriente en la dirección adecuada.

Su color depende del material con el que está fabricado (rojo, verde, azul, blanco, etc.).

Consume muy poca energía y tiene una larga vida útil.

La intensidad de la luz puede controlarse variando la corriente o usando técnicas como PWM (modulación por ancho de pulso).

Planteamiento del proyecto

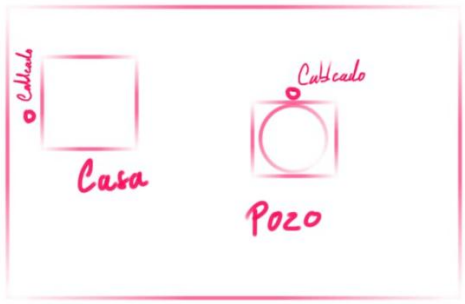
Para este proyecto, se desarrollará una maqueta donde represente su uso práctico en un área rural donde se tenga acceso a un pozo que, mediante procesos naturales, estaría variando su contenido de agua. De esta manera se usará este sensor Sensor Ultrasónico HC-SR04 para establecer parámetros que indiquen qué tan vacío o lleno esté el pozo en cuestión, así como enviar señales a la vivienda cercana encargada del pozo, dentro de un radio de 137 metros.

El radio de los 137 metros es debido a las pruebas anteriormente realizadas donde se comprobaría el alcance de las antenas conectadas a los sistemas embebidos empleados para esta práctica,

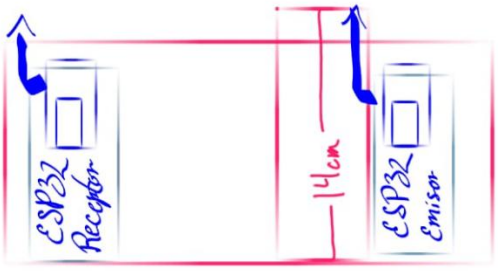
Se espera, entonces, que el sensor envíe de manera segura los datos al receptor que comprende al actuador, un diodo emisor de luz.

Este diodo convierte la señal eléctrica procesada por el ESP32 en una salida física perceptible, es decir, luz. La variación de brillo del LED representa el estado del pozo: que sea más brillante significará que está más lleno; si es menos brillante, está más vacío.

El proyecto contendrá lo siguiente para armar el escenario a escala:

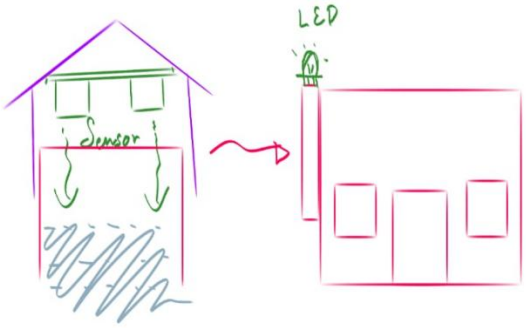


El recuadro es el límite del área trabajada que contendrá el pozo a medir y la vivienda donde se reflejarán los resultados.



Esta sería la vista frontal del modelo, donde se podrá apreciar y confirmar el estado y nivel del pozo para corroborar los resultados del sensor.

También la posición de los procesadores que harán funcionar el escenario.

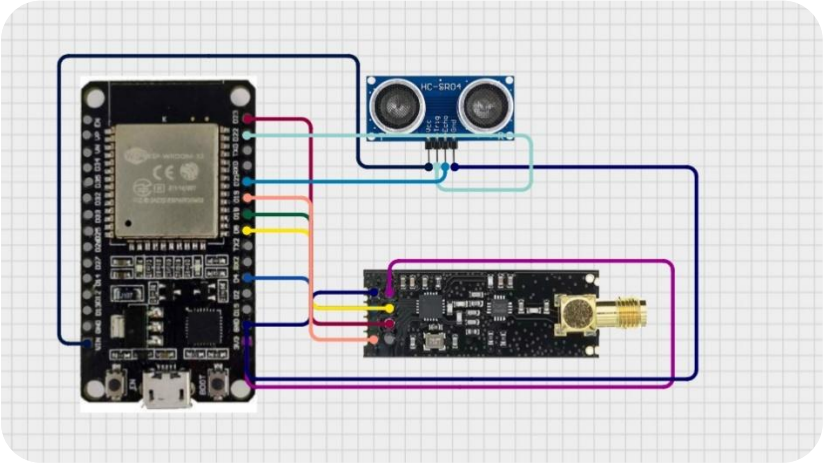


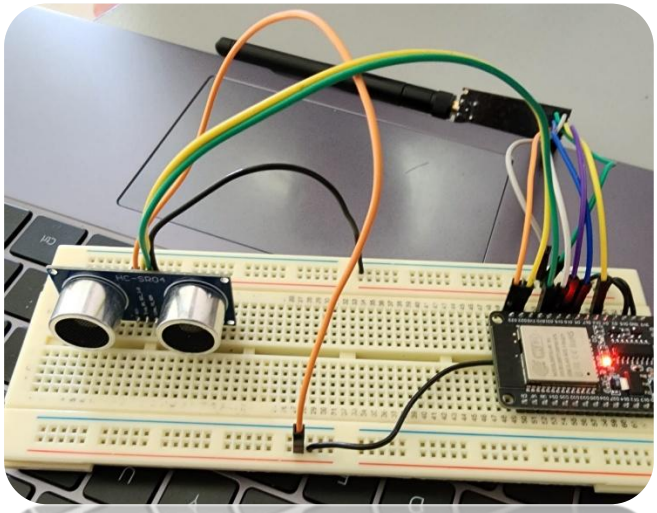
Este diagrama solo refleja brevemente el comportamiento de la información en este sistema.

Circuitos

1. Transmisor

a) Diseño





Este circuito conecta el ESP32 con la antena y tras eso, al sensor HC-SR04, cuidando la posición de los pines para evitar que se quemase la antena y que hiciera la conexión apropiada con los pines del sensor. Ningún componente resultó dañado.

El equipo funcionó adecuadamente y no se presentaron demasiados problemas para obtener los resultados escalables.

b) Código

Librerias que permiten la comunicación de los NRF24L01 que facilita su manejo y librería para encriptar datos con cifrado AES.

```
#include <SPI.h>

#include <RF24.h>

#include "mbedtls/aes.h"
```

Aqui se define los pines del ESP-32 para controlar el módulo NRF24L01, el sensor ultrasónico e inicia una variable para guardar la distancia que lee el sensor

```
#define CE_PIN 4

#define CSN_PIN 5

#define echo 21

#define trigger 22

int distancia;
```

Se crea el objeto radio con los pines definidos y se crea una dirección de comunicación que se usara para ambos dispositivos

```
RF24 radio(CE_PIN, CSN_PIN);

const byte direccion[6] = "00001";
```

Clave AeS de 128 bits para encriptar los datos.

```
const char* clave_segura = "SistemasI_2026_X";
```

Metodo de encriptacion. Configura la clave para la encriptación, aplica el cifrado AES en modo ECB como texto simple y se guardara el cifrado en la variable outputbuffer.

```
void encrypt(unsigned char * plainText, unsigned char * key, unsigned char * outputBuffer) {

  mbedtls_aes_context aes;
```

```
  mbedtls_aes_init(&aes);
  mbedtls_aes_setkey_enc(&aes, (const unsigned char*) key, 128);
  mbedtls_aes_crypt_ecb(&aes, MBEDTLS_AES_ENCRYPT, (const unsigned char*)plainText,
outputBuffer);
  mbedtls_aes_free(&aes);
}
```

Este metodo hace funcionar el sensor ultrasónico, conforme la información llegue mediante los pines del echo y trigger, activa el pin del trigger para que emita el sonido ultrasónico y posteriormente active el pin del echo para escuchar el eco del sonido ultrasónico y devuelve la distancia captada

```
long readUltrasonicDistance(int triggerPin, int echoPin){
  pinMode(triggerPin, OUTPUT);
  digitalWrite(triggerPin, LOW);
  delayMicroseconds(2);
  digitalWrite(triggerPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(triggerPin, LOW);
  pinMode(echoPin, INPUT);
  return pulseIn(echoPin, HIGH);
}
```

Se configurara el setup con una comunicación serial en 115200 y se inicia el pin para controlar el boton

```
void setup() {
Serial.begin(115200);
 pinMode(BOTON_PIN, INPUT_PULLUP);
```

Esta sentencia inicia el modulo NRF24L01

```
if (!radio.begin()) {
Serial.println("NRF24 NO detectado");
while (1);
}
```

Se configuran valores para la comunicación de los módulos. Se usara el canal 69 en 2.4 GHz, configurado para que use la máxima distancia posible y que use una velocidad de 250KBps paratener una transmisión mas confiable y estable.

```
radio.setChannel(69);
radio.setPALevel(RF24_PA_MAX);
radio.setDataRate(RF24_250KBPS);
```

Se abre el canal detransmision y se active el modulo para que funcione como transmisor en modono escuchar.

```
radio.openWritingPipe(direccion);
radio.stopListening();
Serial.println("Transmisor listo (ESP32)");
Serial.println("Transmisor con Encriptación Listo");
}
```

Se hace un calculo para que la distancia que devolvió el método del sensor ultrasónico se convierta en una distancia en cm

```
void loop() {
  distancia = 0.01723 * readUltrasonicDistance(trigger, echo);
```

Se crean dos variables sin asignar para almacenar los datos de entrada y los datos cifrados y se almacena distancia dentro de la variable bufferEntrada

```
  unsigned char bufferEntrada[16] = {0};
  unsigned char bufferCifrado[16] = {0};
  memcpy(bufferEntrada, &distancia, sizeof(distancia));
```

Utiliza el método encrypt para cifrar los datos de bufferEntrada, el resultado se guardara en bufferCifrado

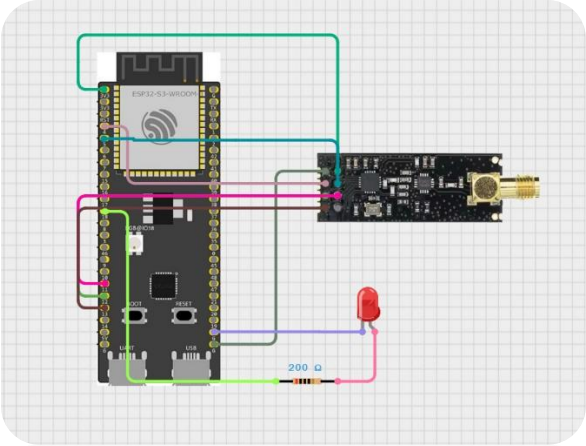
```
  encrypt(bufferEntrada, (unsigned char*)clave_segura, bufferCifrado);
```

Se envia la variable con radio write y si se envia de forma correcta devuelve un mensaje de confirmación, si hay algún error devuelve mensaje de error

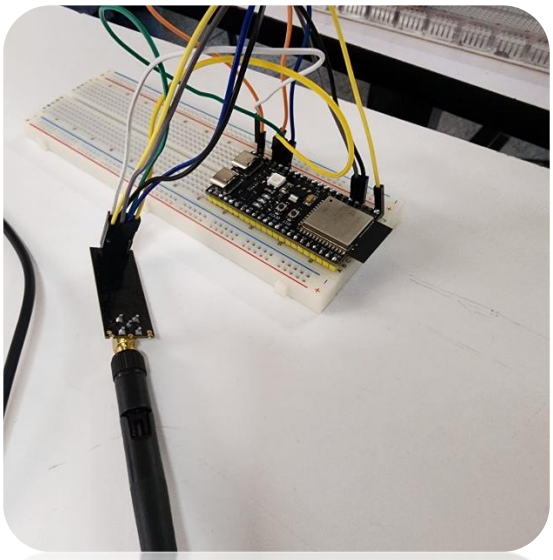
```
  if (radio.write(bufferCifrado, 16)){
    Serial.print("Enviado (Cifrado): ");
    Serial.println(distancia);
  } else {
    Serial.println("Error de envío");
  }
  delay(50);
}
```

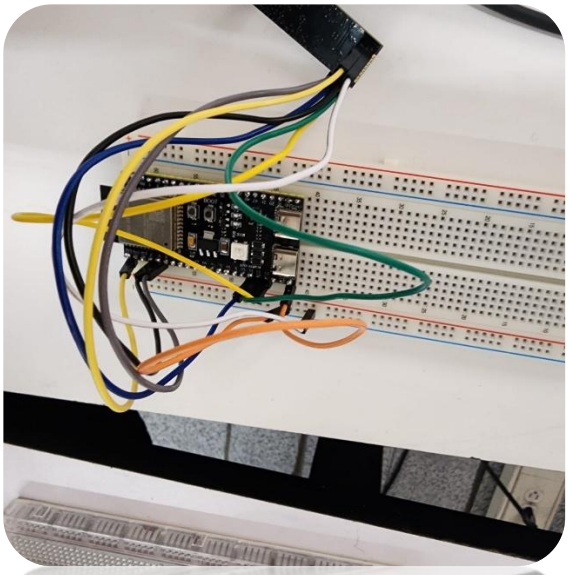
2. Receptor

c) Diseño



Este circuito enlazó de manera similar una antena y un led con el ESP32. De esta forma, el LED integrado en el ESP32 funciona como corroborador de la conexión entre los ESP32 y el LED adicional, para representar la respuesta con el sensor.





d) Código

Librerias que permiten la comunicación de los NRF24L01 que facilita su manejo y librería para cifrar los datos mediante cifrado AES.

```
#include <SPI.h>

#include <RF24.h>

#include "mbedtls/aes.h"
```

Aqui se define los pines del ESP-32 para controlar el módulo NRF24L01 y define un pin para controlar el LED

```
#define CE_PIN 4

#define CSN_PIN 5

#define SCK_PIN 12

#define MISO_PIN 13

#define MOSI_PIN 11

#define pinLed 18
```

Variables para almacenar la distancia recibida y definir la intensidad del LED

```
int brillo =0;

int distancia = 0;
```

Se crea el objeto radio con los pines definidos y se crea una dirección de comunicación que se usara para ambos dispositivos

```
RF24 radio(CE_PIN, CSN_PIN);

const byte direccion[6] = "00001";
```

Clave AES que decrifra los datos

```
const char* clave_segura = "SistemasI_2026_X";
```

Metodo para desencriptar, prepara el contexto AES, utiliza una llave de 128 bits, se guarda el resultado en outputbuffer y libera la memoria al finalizar

```
void decrypt(unsigned char * cipherText, unsigned char * key, unsigned char * outputBuffer) {
  mbedtls_aes_context aes;
  mbedtls_aes_init(&aes);
  mbedtls_aes_setkey_dec(&aes, (const unsigned char*) key, 128);
  mbedtls_aes_crypt_ecb(&aes, MBEDTLS_AES_DECRYPT, (const unsigned char*)cipherText, outputBuffer);
  mbedtls_aes_free(&aes);
}
```

Se configurara el setup con una comunicación serial en 115200, booleano que controla el encendido del led

```
void setup() {
 Serial.begin(115200);
 bool encendido = false;
```

Se inicia la comunicación SPI con los pines que se definieron

```
 SPI.begin(SCK_PIN, MISO_PIN, MOSI_PIN);
```

Esta sentencia inicia el modulo NRF24L01, si no se inicia de forma correcta da un mensaje de error.

```
if (!radio.begin()) {
    Serial.println("NRF24 NO detectado");
    while (1);
```

```
 }
```

Se configuran valores para la comunicación de los módulos. Se usara el canal 69 en 2.4 GHz, configurado para que use la máxima distancia posible y que use una velocidad de 250KBps para tener una transmisión más confiable y estable.

```
 radio.setChannel(69);
 radio.setPALevel(RF24_PA_MAX);
 radio.setDataRate(RF24_250KBPS);
```

Abre el canal de comunicación con una dirección definida y activa el modo de escucha y manda un mensaje sobre el estado del receptor.

```
 radio.openReadingPipe(0, direccion);
 radio.startListening();
Serial.println("Receptor con Desencriptación Listo");
}
```

Si hay datos que haya recibido el modulo NRF25L10, crea dos variables sin asignar para almacenar los datos recibidos y los datos descifrados.

```
Void loop() {
if (radio.available()) {
   unsigned char bufferRecibido[16];
   unsigned char bufferDescifrado[16];
```

Almacena los datos en la variable y utiliza la función decrypt para desencriptar los datos recibidos.

```
   radio.read(bufferRecibido, 16);
   decrypt(bufferRecibido, (unsigned char*)clave_segura, bufferDescifrado);
```

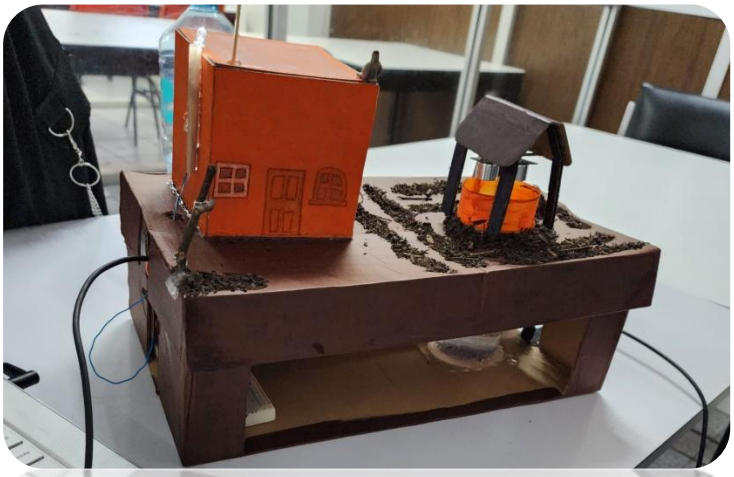
Copia los datos del mensaje desencriptado y los almacena en distancia. Posteriormente manda un mensaje de confirmación con la distancia y cambia el led a encendido

```
    memcpy(&distancia, bufferDescifrado, sizeof(distancia));
    Serial.print("Descifrado correctamente: ");
    Serial.println(distancia);
    encendido = !encendido;
}
```

Se hace un map para controlar la intensidad del LED conforme la distancia que recibe mediante la transmisión de datos, siendo 255 cuando más cercana la distancia y apagándose cuando esta fuera del rango de 50 cm.

```
brillo = map(distancia, 0, 50, 255, 0);
  brillo = constrain(brillo, 0, 255);
  if (distancia > 0) {
    analogWrite (pinLed, brillo);
  }
  delay(10);
```

Diseño final de la maqueta:



Los resultados fueron satisfactorios al momento de utilizar los sensores y actuadores.

Si bien el actuador siempre estuvo encendido pues, en escala, el led ya encendía a su máxima capacidad a menos de 15cm, se comprobó que seguía funcionando adecuadamente si se apuntaba el sensor al aire. Sin duda este proceso actuaría bien en un escenario real, pues hay más espacio.

Los resultados enviaban la distancia en centímetros que habría entre el nivel del pozo con el sensor, por eso, en el video los resultados demostrarían números cada vez más pequeños.

Conclusiones

Ramírez Rodríguez Martín de Jesús:

El sistema implementado demostró ser funcional y confiable en la simulación de un escenario real. El sensor ultrasónico permitió obtener lecturas claras del nivel del pozo y el LED actuó correctamente como indicador visual, aun con las limitaciones de escala de la maqueta. Se comprobó que la lógica de comunicación y actuación responde de manera adecuada, lo que valida la viabilidad del diseño para aplicaciones prácticas en entornos más amplios.

Santacruz Rodarte Cristian Alexis:

Con este proyecto pudimos usar los conocimientos previos para emplear los usos de redes en ESP32 y NRF para un proyecto que puede ser usado para el trabajo y la vida cotidiana, en este caso el manejo de líquidos en un pozo a través de un sensor para la correcta obtención de información..

Pedroza Pérez Emmanuel:

En este proyecto aplicamos nuestros conocimientos adquiridos a lo largo del semestre de tecnologías inalámbricas. Vimos como esos conocimientos nos ayudan a resolver problemáticas de la vida diaria y ayudan a hacer más sencillas ciertas tareas.

Enlace a Video Demostración

https://drive.google.com/drive/folders/1yxoWwSM7txI9WxwEstTsoSHF_w9ZULSf?usp=sharing

Enlace a GitHub

Este enlace que se muestra a continuación, muestra tanto el repositorio donde se encuentra la documentación misma, los códigos empleados para esta práctica y su video de evidencia:

https://github.com/MartinITAAgs/Evaluativo-2---NRF24l01---Sistema-de-control